You can also optionally specify file permissions when creating a file. After opening the file, we can write and/or read data as needed and as specified by the mode string. When we are done, as responsible software citizens, we close the file, ensuring that all buffered data is written and that all related resources are freed.

But Ruby can make life a little bit easier for you. The method File.open also opens a file. In regular use, it behaves just like new. However, if you associate a block with the call, open behaves differently, see Using Blocks for Transactions, on page 72. Instead of returning a new File object, it invokes the block, passing the newly opened File as a parameter. When the block exits, the file is automatically closed.

```
File.open("testfile", "r") do |file|
  # ... process the file
end    # <- file automatically closed here
```

Using File.open with a block has an added benefit. When using File.new as we did earlier, if an exception is raised while processing the file, the call to file.close may not happen. Once the file variable goes out of scope, then garbage collection will eventually close it, but this may not happen for a while. Meanwhile, resources are being held open.

This doesn't happen with the block form of File.open. If an exception is raised inside the block, the file is closed before the exception is propagated on to the caller. It's as if the open method looks like the following:

```
class File
  def self.open(*args)
    f = File.new(*args)
    result = f
    if block_given?
      begin
        result = yield f
      ensure
        f.close
      end
    end
    result
  end
end
```

Reading and Writing Files

The same methods that we've been using for "simple" I/O from standard input and output are available for File objects. So, where Kernel#gets reads a line from standard input (or from any files specified on the command line when the script was invoked), File#gets reads a line from the file object.

For example, we could create a program called copy.rb:

tut_io/copy.rb

```
while (line = gets)
  puts line
end
```

If we run this program with no arguments, it will read lines from the console and copy them back to the console. Note that each line is echoed once the Return key is pressed. (In this and